

SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

1. Introduction:

Medi-Sync+ is a strategic multi-agent orchestrator-based mobile application designed to bridge the gap in post-discharge patient care. It acts as a "virtual nurse," converting fragmented discharge notes into a continuous, intelligent care workflow.

1.1. Purpose:

This system analysis document highlights the technical scope, multi-agent design decisions and architectural framework behind the Medi-Sync+ development, focusing on a multi-agentic orchestrator model. So, The key system elements that have been covered includes:

1. Architecture:

- a. It's a multi-agent orchestrator system where specialized agents (Medicine, Follow-up, Symptoms, etc.) handle distinct medical domains.
- b. Deployed as a cloud-native platform where a central brain coordinates between sub-agents to provide continuous care.

2. Data Flows:

- a. How discharge data flows from historical hospital databases into the orchestrator.
- b. How real-time patient inputs like symptoms and vitals move through the backend API to update the follow-up database.

3. Model Process:

- a. The model shows the end-to-end workflow of converting fragmented discharge notes into a structured and active care plan.
- b. The process of tracking medication adherence, evaluates symptoms and escalates critical risks to the care team or emergency call when signs of deterioration appear.

4. Role of Reference:

- a. This document serves as a high-level architectural guide for the development team and testers to ensure modular interaction between the Symptoms, Medicine and Risk Trajectory agents.

1.2. Background:

Hospitals currently face significant visibility gaps after a patient is discharged, leading to fragmented care and high readmission rates. Patients often suffer from information overload, causing them to forget medication or misunderstand follow-up instructions.

1. Previous Version:

- a. The existing process is entirely manual and unstructured, relying on physical "Discharge Summaries" and verbal advice.
- b. Patients rely on standard mobile alarms or family members to remember care tasks, which are often ignored due to daily distractions.
- c. One-way communication where hospitals have no real-time way to track patient recovery once they leave the ward.

2. Changes in Major Architectural Components:

- a. Implementation of a "Closed-Loop" system that monitors, evaluates, updates, and intervenes automatically .
- b. Transition from a static reminder app to a multi-agent framework where the orchestrator manages specialized reasoning tasks.

3. New Capabilities:

- a. Multi-agent reasoning that predicts health risk trajectory over time rather than just providing one-time alerts.
- b. Automated conversion of hospital discharge notes into a structured and tidy digital care plan.

1.3. Target Stakeholder

Stakeholders	Roles	Expectations
Discharged Patients	Browse medication schedules, report symptoms, and complete daily checklists.	Simple, elderly-friendly UI; clear medication reminders; and immediate

		assistance for risky symptoms.
Hospital Staff	Monitor patient recovery and receive automated alerts for dangerous symptoms.	Structured patient recovery data; reduced readmission rates; and improved visibility post-discharge.
Development Team	Build the orchestrator system and maintain agent logic	Clear multi-agent communication protocols; modular codebase for specific agents
QA Team	Validate system behavior, agent reasoning and critical medical alert	Defined edge cases for symptom detection; prompt response reliability for LLM calls.

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	Details
System	Android Mobile Application
Architecture	Multi-Agent Orchestrator with actual database

MediSync+ is structured as a client-server based system where the mobile client interacts with a complex multi-agent backend. The system incorporates independent agents including a Medicine agent, Follow-up

agent, Symptoms agent, Check list agent, Chat agent, Discharged agent and Risk Trajectory agent . All agents are coordinated by a central Orchestrator. The system communicates with two primary databases: a Historical Ward Database and a Post-Discharge Follow-up Database.

2.1.2 LLM as Service Layer

The architecture integrates GLM as the core reasoning engine for the multi-agentic workflow. The following diagram includes:

2.1.2.1 Dependency Diagram

- a. **Prompt Construction:** Prompts are built by extracting data from hospital discharge notes to form structured care plans.
- b. **Context Window:** The window includes historical ward data, current patient vitals (weight, BP), and real-time symptom descriptions.
- c. **Parsing & Passing:** The system parses LLM outputs to generate specific "MediSync Actions" such as booking appointments or alerting nurses.
- d. **Limitations:** Input data from long medical histories are chunked before being processed by the GLM API to ensure token efficiency.
- e. **Major API Calls:** The mobile app calls the Orchestrator API which then delegates sub-tasks to specialized agent modules .
- f. **Interactions:** The Medicine agent communicates with the Medication database to ensure timing persistence.
- g. **Distributed Components:** The Risk Trajectory agent operates across multiple interaction sessions to detect trends over time.

2.1.2.2 Sequence Diagram

- a. **User interaction flow:** Patient misses medication -> Risk Trajectory agent detects a lapse -> System prompts user for symptoms -> Alert escalated to doctor.
- b. **Step-by-step:** Interaction starts with a notification reminder , followed by patient data entry into the "Today's Checklist".
- c. **Demonstration:** Key features operate by continuously monitoring patient state, evaluating risks,

and updating the follow-up plan .

2.2. Technological Stack

2.2.1. Frontend: Native Android (Kotlin + Jetpack Compose) providing a clean, professional, hospital-themed UI with high-contrast colours and large readable fonts optimised for elderly users. Navigation is handled via a bottom navigation bar across four primary screens: Home, Medication, Vitals, and Checklist.

2.2.2. Backend: On-device multi-agent orchestration implemented in Kotlin using WorkManager (for background workers: MedicationReminderWorker, DailyChecklistGeneratorWorker, EmrSyncWorker) and Hilt for dependency injection. The AgentOrchestrator coordinates all specialized agents (MedicineAgent, FollowUpAgent, SymptomAgent, ChecklistAgent, ChatAgent, RiskTrajectoryAgent, DischargeAgent) within the Android process. LLM reasoning is delegated to the Z AI GLM cloud API via LlmApiService.

2.2.3. Database: Room Database (SQLite on-device) structured in separate entity tables for Patients, Medications, MedicationLogs, Vitals, Appointments, Symptoms, Checklists, CareTeam, and ChatMessages. Bi-directional hospital sync is achieved via EmrSyncWorker using exponential backoff. The on-device schema is designed in 3NF with foreign key constraints enforced at the Room layer.

2.2.4. Cloud/Deployment: Distributed as a standalone Android APK (minimum SDK 26, target SDK 35). Cloud dependencies are limited to: (1) Z AI GLM API for all LLM reasoning calls, accessed via secure HTTPS with BuildConfig-injected API keys that are never exposed in source; and (2) Hospital EHR REST API for bi-directional patient record synchronisation managed by EmrSyncWorker with WorkManager retry policies.

2.3. Key Data Flows

2.3.1 Data Flow Diagram (DFD)

- a. Illustrates data moving from discharge notes into the orchestrator agent and the Orchestrator assigns tasks to specific agents, which then update the follow-up database or trigger external notifications.
- b. Data flows from symptom input screens back into the hospital database to ensure "closed-loop" care.

2.3.2. Normalized Database Schema

- a. **ERD Design:** Database designed in 3NF to manage relationships between Patients, Medications, and Medical Alerts.
- b. **Junction Tables:** A Patient-Symptom table to track various vitals like weight, blood pressure, and blood sugar level over time.

3. Functional Requirements & Scope

3.1. Minimum Viable Product:

3.1.1. Core Features

#	Features	Description
1	Medicine Management	Provides medication details, UI schedules, and notification reminders for patients.
2	Follow-up Visit Tracking	Tracks future appointment schedules and sends automated visit reminders.
3	Today's Checklist	Daily vitals logging (BP, weight) with pop-out windows for record entry.
4	Condition Analysis	Agents analyze symptom descriptions for dangerous signs and alert users accordingly.

5	Reasoning Chatbot	Users ask details about medications or health advice; responses are grounded in the medical database.
---	--------------------------	---

3.2. Non-Functional Requirements (NFRs)

Quality (e.g.)	Requirements (e.g.)	Implementation (e.g)
Reliability	Placement of medical alerts must have a 99.9% uptime to ensure patient safety.	Use of multiple specialized agents coordinated by a central orchestrator to avoid single-point reasoning failure.
Maintainability	Each medical domain agent (Medicine, Follow-up, Symptoms) is separated and deployed individually .	Modular agent architecture allows for updates to medical knowledge without impacting the core orchestrator.
Token Latency	API calls to the GLM for symptom analysis must return a response within 3s under normal load.	Async API calls with a user-facing loading state while agents perform multi-step reasoning.
Accessibility	The system must be usable by elderly and low-income patients.	Simple, elderly-friendly interface design with high-contrast, large fonts.

3.3. Out of Scope / Future Enhancements

- Real-time integration with wearable devices for automated heart rate and sleep tracking.
- A dedicated family portal allowing caregivers to monitor patient adherence remotely.

- c. Multi-language support for diverse patient demographics in different regional settings.

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

4.1.1. Grayscale Rollout:

Release the MediSync+ feature to 5% of chronic disease patients initially, monitoring for agent accuracy in symptom flagging

4.1.2. Emergency Rollback:

If symptom detection anomalies exceed a threshold, the system triggers ER and deploys a version with simplified, rule-based medical reminders .

4.1.3. Priority Matrix:

P1 Critical: If dangerous symptoms are detected , an immediate system alert is escalated to the hospital dashboard within 10 seconds.

4.2. Monitoring:

Real-time health monitoring tracks agent reasoning performance and ensures all medication notifications are delivered on schedule.

4.3. Assumptions:

- a. Discharged patients have access to a smartphone with stable internet for agent communication.
- b. Hospital staff are available to receive and act upon high-priority risk alerts generated by the system.

4.4. External Dependencies:

Tools	Purpose	Risks
Z.AI GLM API	Core LLM reasoning engine for multi-step symptom analysis and structured output.	High risk if the API is unavailable, the core “virtual nurse” reasoning will fail.

Hospital DB API	Fetching historical ward records and patient discharge instructions.	Medium risk relies on hospital IT infrastructure and secure access protocols
------------------------	--	--

5. Project Management & Team Contributions

5.1 Project Timeline

- **Day 1:** Brainstorming ideas
- **Day 2 - 3:** Project Planning, Stakeholder analysis and Problem Statement validation
- **Day 4 - 6** Development of the multi-agent orchestrator, database setup and UI/UX design.
- **Day 6 :** seek mentorship, idea rejected
- **Day 7 :** Brainstorming for another idea
- **Day 8 :** Project Planning, Stakeholder analysis and Problem Statement validation
- **Day 9 -11:** Development of the multi-agent orchestrator, database setup and UI/UX

5.2 Team members roles

- **Toh Bin Bin — Project Manager & lead Architect**

Responsible for the high-level design of the multi-agentic orchestrator system that manages the patient discharge process. They coordinate the specialized sub-agents—including the Medicine, Follow-up, and Symptoms agents—to ensure the system provides a continuous care workflow rather than fragmented, reactive care .

- **Yong Jia Khen — AI & Prompt Engineer**

Specializes in prompt construction and context window management for the GLM API to facilitate multi-step reasoning . They are responsible for developing the logic for the Risk Trajectory Agent, which analyzes symptoms over time to detect dangerous trends and predict health risks.

- **Yee Feng Teing — Mobile UI/UX Developer**

Focused on building the Android mobile application with an emphasis on a clean, professional, and elderly-friendly interface . Their primary responsibility includes designing the medication reminder interface, symptom reporting screens, and the "Today's Checklist" with pop-out windows for vital record entry .

- **Louis Chua Kai Yi — Backend & Database Engineer**

Responsible for implementing the microservices and managing the actual databases, including the "Patient in Ward" historical database and the "Patient Follow-up" database . They ensure proper tool calls for data persistence, such as recording reported symptoms and medical vitals to the hospital system for doctor visibility.